Made with Goodnotes

COAL Assignment 01

NAME : Hania Waqar ROLL NO : 25K-0022

CLASS : BAI - 3A

=D Question 01 :

```
mov ax, 3
mov bx, ax
shl ax, 4
add ax, bx
mov bx, 3
shl bx, 1
add ax, bx
```

=D Question 02 :

```
.data
  num DWORD 02345ABCDh
.code
  mov esi, OFFSET num
  mov ecx, 4
 Right shift:
   ror BYTE PTR [esi], 1
   inc esi
   loop Rightshift
 mov esi, OFFSET num
 mov ecx, 4
 Left shift:
    rol BYTE PTR [esi], 1
    inc esi
    loop Leftshift
```

Made with Goodnotes

=D Question 03:

```
.data
 num DWORD  0234SABCDh
.code
 mov esi, OFFSET num
 mov ecx, 2
Right Shift:
    ror WORD PTR [esi], 1
    add esi, 2
    loop Rightshift
 mov esi, OFFSET num
 mov ecx, 2_
Left Shift:
 rol WORD PTR [esi], 1
 add esi, 2
 loop LeftShift
```

=D Question 4:

→ Part (a):

```
.data
 Bseconds BYTE ?
.code
 mov dx, 0101100111011010b
 mov ax, dx
 and ax, 001Fh
 mov Bseconds, al
```

→ Part (b):

```
.data
 Bminutes BYTE ?
.code
 mov dx, 0101100111011010b
```

Made with Goodnotes

```
mov ax, dx
shr ax, 5
and ax, 003Fh
mov Bminutes, al
```

=> Question 05:

Instruction	Ax	Dx	CF
mov dx, 9A3Fh	-	9A3Fh	-
mov ax, 7580h	7580h	9A3Fh	-
clc	7580h	9A3Fh	0
add ax, 90EAh	106Ah	9A3Fh	1
adc dx, 1	106Ah	9A41h	0
stc	106Ah	9A41h	1
add ax, 8120h	918Ah	9A41h	0
adc dx, 0FFFFh	918Ah	9A40h	1

=> Question 6:

(a) TEST ax, ax

(b) TEST bx, 1

(c) TEST dx, 8000h

(d) TEST dx, 8000h

(e) TEST bl, bl

=> Question 7:

```
mov dx, 5
stc
mov ax, 10h
adc dx, ax
```

Made with Goodnotes

⇒ Question 8 :

Original instruction	Problem	Correction
mov esi, val 1	loads the val not its address	mov esi, OFFSET val 1
mov edi, val 2	loads the val not its address	mov edi, OFFSET val2
mov BYTE PTR [esi], al	overwrites val 1 instead of result	store through a result pointer
dec esi	moves backward	inc esi
dec edi	moves backward	inc edi
no result pointer	cannot store into result	initialize EBX to result address
loop top with SBB	loop itself preserve CF, but the loop must not insert flag changing arithmetic b/w byte subtractions.	use INC for pointer updates

⇒ Question 09 :

```
INCLUDE Irvine32.inc
.data
 msg1 BYTE " Enter marks (0-100): ", 0
 msg2 BYTE "Grade: ", 0
 invalid BYTE "Invalid marks: ", 0
.code
 main PROC
      mov edx, OFFSET msg1
      call WriteString
      call ReadInt
      cmp eax, 0
      jl Invalidmarks
      cmp eax, 100
      jg Invalid marks
      mov edx, OFFSET msg
      call writestring
      cmp eax, 80
```

Made with Goodnotes

```
    jae GradeA
    cmp eax, 60
    jae GradeB
    cmp eax, 40
    jae GradeC
    jmp GradeF
GradeA:
    mov al, 'A'
    jmp display
GradeB:
    mov al, 'B'
    jmp display
GradeC:
    mov al, 'C'
    jmp display
GradeF:
    mov al, 'F'
    jmp display
Display:
    call WriteChar
    call Crlf
    jmp Finish
Invalid marks:
    mov edx, OFFSET invalid
    call WriteString
    call Crlf
Finish:
    exit
main ENDP
END main
```

Made with Goodnotes

=> Question 10:

Instruction	EAX	EBX	Stack
mov eax, 3	3	–	empty
mov ebx, 7	3	7	empty
push eax	3	7	[3]
push ebx	3	7	[7, 3]
pop eax	7	7	[3]
push 10	7	7	[10, 3]
pop ebx	7	10	[3]
pop eax	3	10	empty

=> Question 11:

```
.data
    num1 SDWORD ?
    num2 SDWORD ?
    num3 SDWORD ?
    num4 SDWORD ?
    result SDWORD ?
.code
    main PROC
        mov eax, num1
        cmp eax, num2
        jbe Elseif
        mov eax, num3
        cmp eax, num4
        je Elseif
        mov eax, num1
        add eax, num3
        mov result, eax
        jmp Endif
```

Made with Goodnotes

```
Elseif:
    mov eax, num2
    cmp eax, num4
    jne ElsePart
    mov eax, num2
    sub eax, num1
    mov result, eax
    jmp Endif
ElsePart:
    mov result,0
Endif:
    ret
main ENDP
END main
```

⇒ Question 12:

```
INCLUDE Irvine32.inc

.data
    op1  SDWORD  ?
    op2  SDWORD  ?
    x    SDWORD  ?
    y    SDWORD  ?
    z    SDWORD  ?
.code
main PROC
  outerloop:
        mov eax, op1
        cmp eax, op2
        jg Done
```

Made with Goodnotes

```
Inner Loop:
    mov eax, op1
    cmp eax, op2
    jg EndInner
    cmp eax, x
    jl ElseBlock
    cmp eax, y
    jg ElseBlock
    add z, 10
    jmp ContinueLoop
ElseBlock:
    sub z, 10
Continue Loop:
    dec op1
    jmp innerLoop
EndInner:
    jmp outerloop
Done:
    exit
main ENDP
END main
```

Made with Goodnotes